

Introduction

For each textual review, we want to predict if it corresponds to a good review (the customer is happy) or to a bad one (the customer is not satisfied). The reviews overall ratings can range from 2.5/10 to 10/10. In order to simplify the problem we will split those into two categories:

bad reviews have overall ratings good reviews have overall ratings

Load data

We first start by loading the raw data. Each textual reviews is splitted into a positive part and a negative part. We group them together in order to start with only raw text data and no other information.

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)

```
cd /content/drive/MyDrive/shilpa
```

```
/content/drive/MyDrive/shilpa
```

```
import pandas as pd

# read data
reviews_df = pd.read_csv("Hotel_Reviews.csv")
# append the positive and negative text reviews
reviews_df["review"] = reviews_df["Negative_Review"] + reviews_df["Positive_Review"]
# create the label
reviews_df["is_bad_review"] = reviews_df["Reviewer_Score"].apply(lambda x: 1 if x < 5 else 0)
# select only relevant columns
reviews_df = reviews_df[["review", "is_bad_review"]]
reviews_df.head()
```

	review	is_bad_review
0	I am so angry that i made this post available...	1
1	No Negative No real complaints the hotel was g...	0
2	Rooms are nice but for elderly a bit difficul...	0
3	My room was dirty and I was afraid to walk ba...	1
4	You When I booked with your company on line y...	0

Next steps: [Generate code with reviews_df](#) [New interactive sheet](#)

```
reviews_df = reviews_df.sample(frac = 0.1, replace = False, random_state=42)
reviews_df["review"] = reviews_df["review"].apply(lambda x: x.replace("No Negative", "").replace("No Positive", ""))
```

Sample data

Reviews data is sampled in order to speed up computations

Clean data

If the user doesn't leave any negative feedback comment, this will appear as "No Negative" in our data. This is the same for the positive comments with the default value "No Positive". We have to remove those parts from our texts.

The next step consists in cleaning the text data with various operati

```
import nltk
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger')
nltk.download('wordnet')
nltk.download('vader_lexicon')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] /root/nltk_data...
[nltk_data] Package averaged_perceptron_tagger is already up-to-
[nltk_data] date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package vader_lexicon to /root/nltk_data...
[nltk_data] Package vader_lexicon is already up-to-date!
True
```

To clean textual data, we call our custom 'clean_text' function that performs several transformations:

lower the text tokenize the text (split the text into words) and remove the punctuation remove useless words that contain numbers remove useless stop words like 'the', 'a', 'this' etc. Part-Of-Speech (POS) tagging: assign a tag to every word to define if it corresponds to a noun, a verb etc. using the WordNet lexical database lemmatize the text: transform every word into their root form (e.g. rooms -> room, slept -> sleep) Now that we have cleaned our data, we can do some feature engineering for our modelization part.

```
import nltk

nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger_eng')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /root/nltk_data...
[nltk_data] Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data] date!
True
```

```
# return the wordnet object value corresponding to the POS tag
from nltk.corpus import wordnet

def get_wordnet_pos(pos_tag):
    if pos_tag.startswith('J'):
        return wordnet.ADJ
    elif pos_tag.startswith('V'):
        return wordnet.VERB
    elif pos_tag.startswith('N'):
        return wordnet.NOUN
    elif pos_tag.startswith('R'):
        return wordnet.ADV
    else:
        return wordnet.NOUN

import string
from nltk import pos_tag
from nltk.corpus import stopwords
from nltk.tokenize import WhitespaceTokenizer
from nltk.stem import WordNetLemmatizer

def clean_text(text):
    # Ensure text is a string before processing
    if not isinstance(text, str):
        return ""
    # lower text
    text = text.lower()
    # tokenize text and remove punctuation
    text = [word.strip(string.punctuation) for word in text.split(" ")]
    # remove words that contain numbers
    text = [word for word in text if not any(c.isdigit() for c in word)]
    # remove stop words
    stop = stopwords.words('english')
    text = [x for x in text if x not in stop]
    # remove empty tokens
    text = [t for t in text if len(t) > 0]
    # pos tag text
    pos_tags = pos_tag(text)
    # lemmatize text
    text = [WordNetLemmatizer().lemmatize(t[0], get_wordnet_pos(t[1])) for t in pos_tags]
    # remove words with only one letter
    text = [t for t in text if len(t) > 1]
    # join all
```

```

text = " ".join(text)
return(text)

# clean text data
reviews_df["review_clean"] = reviews_df["review"].apply(lambda x: clean_text(x))

```

Splitting Train Data

Here we are going set the variable attribute to hold the movie review texts, and variable target to hold the conclusion [positive ; negative] of the movie review We will integrate those 4 values as features in our dataset.

```

# add sentiment analysis columns
from nltk.sentiment.vader import SentimentIntensityAnalyzer

sid = SentimentIntensityAnalyzer()
reviews_df["sentiments"] = reviews_df["review"].apply(lambda x: sid.polarity_scores(x))
reviews_df = pd.concat([reviews_df.drop(['sentiments'], axis=1), reviews_df['sentiments'].apply(pd.Series)], axis=1)

```

```

# add number of characters column
reviews_df["nb_chars"] = reviews_df["review"].apply(lambda x: len(x))

# add number of words column
reviews_df["nb_words"] = reviews_df["review"].apply(lambda x: len(x.split(" ")))

```

```

!pip install gensim

# create doc2vec vector columns
from gensim.test.utils import common_texts
from gensim.models.doc2vec import Doc2Vec, TaggedDocument

documents = [TaggedDocument(doc, [i]) for i, doc in enumerate(reviews_df["review_clean"].apply(lambda x: x.split(" ")))]

# train a Doc2Vec model with our text data
model = Doc2Vec(documents, vector_size=5, window=2, min_count=1, workers=4)

# transform each document into a vector data
doc2vec_df = reviews_df["review_clean"].apply(lambda x: model.infer_vector(x.split(" "))).apply(pd.Series)
doc2vec_df.columns = ["doc2vec_vector_" + str(x) for x in doc2vec_df.columns]
reviews_df = pd.concat([reviews_df, doc2vec_df], axis=1)

```

```

Collecting gensim
  Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (8.4 kB)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.1)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.1.1)
Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (27.9 MB)
----- 27.9/27.9 MB 44.7 MB/s eta 0:00:00
Installing collected packages: gensim
Successfully installed gensim-4.4.0

```

Double-click (or enter) to edit

```

# add tf-idfs columns
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(min_df = 20)
tfidf_result = tfidf.fit_transform(reviews_df["review_clean"]).toarray()
tfidf_df = pd.DataFrame(tfidf_result, columns = tfidf.get_feature_names_out())
tfidf_df.columns = ["word_" + str(x) for x in tfidf_df.columns]
tfidf_df.index = reviews_df.index
reviews_df = pd.concat([reviews_df, tfidf_df], axis=1)

```

```
reviews_df.head()
```

	review	is_bad_review	review_clean	neg	neu	pos	compound	nb_chars	nb_words	doc2vec_vector_0	...	word
5013	Small room staff not super helpful	0	small room staff super helpful	0.578	0.422	0.000	-0.6681	35	7	-0.042496	...	
3946	the room was quite small and the windows not ...	0	room quite small window insulate outside noise...	0.000	0.759	0.241	0.8608	185	35	0.246307	...	
516	Air conditioning didn t cope with the really ...	0	air condition cope really hot weather luckily ...	0.000	0.591	0.409	0.9951	505	99	1.029573	...	
6404	The staff were fantastic from the first minut...	0	staff fantastic first minute young couple firs...	0.000	0.673	0.327	0.9911	497	98	0.818865	...	
14992	Friendliness of the staff Great restaurants c...	0	friendliness staff great restaurant close good...	0.000	0.524	0.476	0.8750	89	16	0.062719	...	

5 rows × 42 columns

Exploratory data analysis

reviews_df.shape

(269, 42)

show is_bad_review distribution
reviews_df["is_bad_review"].value_counts(normalize = True)

proportion	
is_bad_review	
0	0.97026
1	0.02974

dtype: float64

Result

```
# wordcloud function

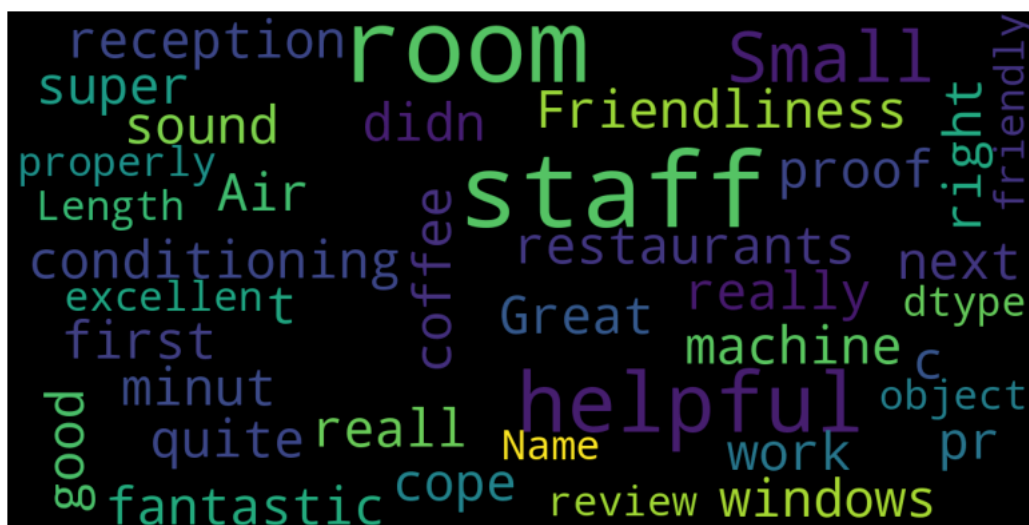
from wordcloud import WordCloud
import matplotlib.pyplot as plt

def show_wordcloud(data, title = None):
    wordcloud = WordCloud(
        background_color = 'black',
        max_words = 200,
        max_font_size = 40,
        scale = 3,
        random_state = 42
    ).generate(str(data))

    fig = plt.figure(1, figsize = (10, 10))
    plt.axis('off')
    if title:
        fig.suptitle(title, fontsize = 10)
        fig.subplots_adjust(top = 2.3)

    plt.imshow(wordcloud)
    plt.show()

# print wordcloud
show_wordcloud(reviews_df["review"])
```



```
# highest positive sentiment reviews (with more than 10 words)
reviews_df[reviews_df["nb_words"] >= 10].sort_values("pos", ascending = False)[["review", "pos"]].head(10)
```

	review	pos	
19163	Amazing view huge comfy bed delicious breakfa...	0.760	
1834	Received a lovely free room upgrade Beautiful...	0.727	
1475	Staff very friendly and helpful Room excellen...	0.722	
19812	Tiny room Ok for one but glad I wasn t sharin...	0.705	
26638	Friendly staff and lovely atmosphere Comforta...	0.658	
3776	Clean fresh well appointed rooms very tidy in...	0.656	
14985	Clean spacious room Friendly staff and good f...	0.638	
5187	Room a bit small Great Location good Value fo...	0.610	
18884	N A Very helpful and friendly staff Quiet roo...	0.587	
18927	Luxurious hotel Great gym and spa facilities ...	0.576	

```
# lowest negative sentiment reviews (with more than 10 words)
reviews_df[reviews_df["nb_words"] >= 10].sort_values("neg", ascending = False)[["review", "neg"]].head(10)
```

	review	neg	
20042	very noisy too expensive considering faciliti...	0.415	
13943	20 minutes to book in spa was not very clean ...	0.352	
16111	Rooms and bathrooms Air condition didn t work...	0.327	
6657	See above Coffee awful bed comfy pillows syn...	0.248	
4501	Eventually when I returned there was only one...	0.242	
13302	rooms were tiny and the kettle was unclean an...	0.236	

plot sentiment distribution for positive and negative reviews

```
import seaborn as sns
```

```
for x in [0, 1]:
    subset = reviews_df[reviews_df['is_bad_review'] == x]

    # Draw the density plot
    if x == 0:
        label = "Good reviews"
    else:
        label = "Bad reviews"
    sns.distplot(subset['compound'], hist = False, label = label)
```

/tmp/ipykernel_235/4231220472.py:13: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `kdeplot` (an axes-level function for kernel density plots).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(subset['compound'], hist = False, label = label)
```

/tmp/ipykernel_235/4231220472.py:13: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `kdeplot` (an axes-level function for kernel density plots).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(subset['compound'], hist = False, label = label)
```

